

IIT JAMMU SUMMER SCHOOL 2026

IoT & Embedded Systems Program

ASSIGNMENT QUESTION BANK

IoT Programming, GitHub Integration &
Hands-On Project Challenges

Course	Internet of Things – Hands-On Learning
Total Assignments	5 Modules 50 Questions 100 Marks
Submission Mode	GitHub Repository + PDF Report

Note to Students: Upload ALL code to GitHub. Submissions without a GitHub link will NOT be evaluated.

Make sure to attach a screenshot of your TinkerCad / Wokwi diagram and Serial Monitor Output with each question.

Get a chance to work with us at Techible and gain hands-on exposure and a letter of recommendation for your future. Additional support will be provided to overachievers

Table of Contents

Module 1	GitHub Setup & Version Control	3
Module 2	IoT Fundamentals & Arduino Programming	5
Module 3	Sensors, Actuators & Data Acquisition	8
Module 4	Wireless Communication & Protocols	11
Module 5	Advanced IoT Projects & Problem Statements	14
Appendix A	GitHub Submission Checklist	18
Appendix B	Grading Rubric	19

Module 1: GitHub Setup & Version Control

Weightage: 20 Marks | Questions: Q1–Q10

Learning Objectives

- Master GitHub repository management for IoT
- Implement Git version control for embedded C++ code
- Create professional project documentation

Section A: Short Answer Questions (2 marks each)

Q1. GitHub Repository Creation

Create a GitHub account and set up a new public repository named `iot-summer-school-2026`. Your repository must include:

- A `README.md` with your name, roll number, and a 3-line project description
- A folder structure: `/week1`, `/week2`, `/week3`, `/final-project`
- A `.gitignore` file configured for Arduino/C++ projects
- An appropriate open-source license (e.g., MIT)

Deliverable: Paste your GitHub repository URL in the submission form.

Q2. Git Workflow – Push Your First Code

Upload the following LED blink sketch to your repository under `/week1/led_blink/`:

```
void setup() {  
    pinMode(13, OUTPUT);  
}  
  
void loop() {  
    digitalWrite(13, HIGH);  
    delay(1000);  
    digitalWrite(13, LOW);  
    delay(1000);  
}
```

Commit this file with a meaningful commit message (e.g., feat: add LED blink starter code). Screenshot your commit history and upload it.

Q3. Branching Strategy

In your repository, create three branches named: dev, sensor-module, and wireless-module. In the dev branch, add a file notes.md describing what each branch will contain. Merge dev into main and show the merge commit in your submission.

Evaluation: GitHub branch graph screenshot + merge commit URL.

Section B: Practical Tasks (4 marks each)

Q4. Versioned Code – Multiple Commits

Starting from your LED blink code (Q2), progressively improve it across 4 Git commits:

1. v1.0 – Basic LED blink (500ms interval)
2. v1.1 – Add serial print: Blink count: X
3. v1.2 – Read a potentiometer to control blink speed
4. v1.3 – Add a comment header block with author name, date, and description

Each commit must have a descriptive message following the conventional commits format (feat:, fix:, docs:, chore:).

Q5. README Documentation

Write a professional README.md for your /week1/led_blink/ project. The README must include: Project Title, Hardware Required (list of components), Circuit Diagram Description (text-based), How to Upload Code (step-by-step), Expected Output, and Troubleshooting Tips (minimum 3 points).

Q6. Issue Tracking & Milestones

Create 3 GitHub Issues in your repository describing hypothetical bugs or features:

- Bug: Blink interval not changing when potentiometer is at maximum
- Feature: Add RGB LED support
- Enhancement: Log data to Serial Monitor in CSV format

Assign each issue to yourself, add appropriate labels, and create a milestone called 'Week 1 Completion'. Close at least one issue with a commit reference (Fixes #1).

Q7. Collaborative Fork & Pull Request

Fork a partner's `iot-summer-school-2026` repository. Add a new file `/week1/blink_modified.ino` that blinks the LED 3 times fast, then pauses for 2 seconds. Submit a Pull Request to your partner's repository with a proper title and description. Your partner must review and merge your PR.

Deliverable: URL of your Pull Request.

Q8. CI/CD with GitHub Actions (Challenge Question)

Set up a basic GitHub Actions workflow (`.github/workflows/lint.yml`) that runs a Python script to count the number of `.ino` files in your repository and prints the count whenever code is pushed. This tests your ability to automate tasks using GitHub Actions.

Q9. (Theory) Explain the difference between `git pull`, `git fetch`, and `git clone`. When would you use each? (2 marks)

Q10. (Theory) What is a `.gitignore` file? Write a `.gitignore` for an Arduino project that excludes compiled output files (`.hex`, `.elf`), OS-specific files (`.DS_Store`, `Thumbs.db`), and IDE config folders (`.vscode/`, `build/`). (2 marks)

Module 2: IoT Fundamentals & Arduino Programming

Marks: 20 | Questions: Q11–Q20 | Type: Code + Explanation



Hardware Required for This Module

- Arduino UNO / Nano / Mega (any one)
- LEDs (Red, Green, Blue), 220Ω resistors, Buzzer
- Push buttons, 10kΩ pull-down resistors
- Breadboard, jumper wires, USB cable

Section A: Conceptual Questions (2 marks each)

Q11. IoT Architecture

Draw and label a 4-layer IoT architecture diagram: Perception Layer, Network Layer, Processing Layer, and Application Layer. Give one example device/technology for each layer. (Hand-drawn photo accepted; upload to GitHub under /theory/)

Q12. Microcontroller vs Microprocessor

Fill in the following comparison table and upload a photo/scan to your GitHub repository:

Parameter	Microcontroller (Arduino UNO)	Microprocessor (Raspberry Pi)
CPU Speed		
On-chip RAM		
On-chip Flash		
Operating System		
Typical Use Case		
Power Consumption		

Q13. Arduino Pin Types

List and explain with examples: Digital Input, Digital Output, Analog Input, PWM Output, and I2C/SPI pins available on Arduino UNO. For each, give one real IoT use case.

Section B: Programming Tasks (4 marks each)

Q14. Traffic Light Controller

Write an Arduino program that simulates a traffic light system using 3 LEDs (Red, Yellow, Green):

- RED ON for 5 seconds → YELLOW ON for 2 seconds → GREEN ON for 4 seconds
- Add a pedestrian button (push button on pin 7): when pressed, force RED immediately and hold for 8 seconds
- Print the current light state to Serial Monitor with timestamp (millis())

Code must be uploaded to: /week2/traffic_light/traffic_light.ino

Q15. Digital Piano using Buzzer

Build a 4-key musical instrument using push buttons and a passive buzzer:

- 4 buttons mapped to notes: Do (262Hz), Re (294Hz), Mi (330Hz), Fa (349Hz)
- Button press plays the note; button release silences the buzzer
- If two buttons are pressed together: play Sol (392Hz) as a chord substitute
- Add a mode toggle (5th button): switch between Major and Minor scale

Commit separately for each feature using Git.

Q16. Serial Command Interface

Write a program that receives commands over Serial Monitor and controls hardware:

```
LED_ON   → Turn built-in LED on
LED_OFF  → Turn built-in LED off
BLINK_X  → Blink LED X times (X is 1-9)
STATUS   → Print pin states
RESET    → Restart blink counter
```

Implement input validation: unknown commands should print 'ERROR: Unknown command'.

Q17. PWM Fading Night Light

Create an automatic night light that fades an LED in and out (breathing effect) continuously.

Implement three modes switchable via a button: (1) Slow breathing (3-second cycle), (2) Fast pulse (0.5-second cycle), (3) SOS pattern (... --- ...). Display current mode on Serial Monitor.

Q18. State Machine Implementation

Implement a simple vending machine state machine with 4 states: IDLE, COIN_INSERTED, ITEM_SELECTED, DISPENSING. Use 3 buttons as inputs (Insert Coin, Select Item, Cancel). Use 3 LEDs to show the current state. Print all state transitions to Serial Monitor. Code must include a comment block documenting the state transition diagram.

Q19. (Theory) What is the difference between analogWrite() and analogRead() in Arduino? What is PWM and why is it used? Give a practical IoT example for each. (2 marks)

Q20. (Theory) Explain the setup() and loop() functions in Arduino. What happens if you put a long delay() inside loop()? How does this affect sensor reading responsiveness? What is the non-blocking alternative? (2 marks)

Module 3: Sensors, Actuators & Data Acquisition

Marks: 20 | Questions: Q21–Q30 | Type: Hands-On Experiments



Sensors Covered in This Module

- DHT11/DHT22 – Temperature & Humidity
- HC-SR04 – Ultrasonic Distance
- LDR – Light Dependent Resistor
- PIR – Passive Infrared Motion Sensor
- Soil Moisture Sensor, MQ-2 Gas Sensor (optional)

Section A: Sensor Experiments (4 marks each)

Q21. Environmental Monitoring Station (DHT11)

Build a mini weather station using DHT11 sensor:

- Read temperature (°C and °F) and humidity every 2 seconds
- Display readings on Serial Monitor in CSV format: timestamp,temp_C,temp_F,humidity
- Trigger a Red LED if temperature > 35°C or humidity > 80%
- Trigger a Green LED if conditions are normal
- Save 5 sample CSV readings in a file /week3/data/sample_readings.csv in your GitHub repo

Library: DHTesp or DHT by Adafruit. Include library version in README.

Q22. Ultrasonic Parking Sensor

Using HC-SR04, build a parking distance alert system:

- Distance > 50cm: No alert, Serial prints SAFE
- Distance 20–50cm: Yellow LED ON, buzzer beeps every 500ms
- Distance 10–20cm: Red LED ON, buzzer beeps every 200ms
- Distance < 10cm: All LEDs flash rapidly, buzzer continuous
- Calculate and display distance in cm using the formula: $\text{distance} = (\text{duration} * 0.034) / 2$

Advanced: Use millis() for non-blocking timing (no delay()). +1 bonus mark.

Q23. Smart Street Light (LDR + PIR)

Implement an intelligent street light controller:

- LDR below threshold (dark): Enable motion detection mode
- PIR detects motion at night: Turn LED on at full brightness for 30 seconds

- No motion after 30 sec: Dim LED to 20% using PWM
- LDR above threshold (daylight): Turn LED completely OFF
- Log every event to Serial: [HH:MM:SS] EVENT: description

Submit a circuit diagram sketch (hand-drawn or Tinkercad screenshot) in `/week3/schematics/`.

Q24. Multi-Sensor Data Logger

Build a system that reads 3 sensors simultaneously and logs structured data:

- DHT11: Temperature, Humidity
- LDR: Light level (0–1023 raw, converted to percentage)
- HC-SR04: Distance in cm

Output format every 5 seconds:

```
=== SENSOR LOG ===
Time       : 12345 ms
Temp       : 28.5 C | Humidity: 65%
Light      : 73% (Bright)
Distance   : 42 cm
=====
```

Upload 20 lines of captured Serial Monitor output as `/week3/data/sensor_log.txt`.

Section B: Actuator Control (2 marks each)

Q25. Servo Motor Sweep

Control a SG90 servo motor with a potentiometer. Potentiometer position (0–1023) maps to servo angle (0°–180°). Display angle in Serial Monitor. Add a button that makes the servo do one full sweep (0→180→0) when pressed. Code: `/week3/servo_control/`

Q26. DC Motor Speed Control with L298N

Control a DC motor through L298N driver: potentiometer sets speed (0–255 PWM), one button changes direction (Forward/Reverse), another button stops/starts the motor. Display Direction, Speed (0–100%), and State on Serial Monitor.

Q27. Relay-Controlled AC Device Simulation

Simulate a relay controlling an AC appliance (use LED as a safe simulation). DHT11 triggers relay ON when temperature > 32°C (AC turns on). Relay OFF when temp drops below 28°C (hysteresis). Add a manual override button. Log all relay state changes with temperature readings.

Q28. Keypad + LCD Display

Using a 4x4 membrane keypad and 16x2 LCD, build a password-protected access system: Display 'ENTER PIN:' on LCD, user enters 4-digit PIN, correct PIN (set in code) shows 'ACCESS GRANTED' + Green LED, wrong PIN shows 'ACCESS DENIED' + Red LED + buzzer. After 3 wrong attempts, lock for 10 seconds.

Q29. (Theory) What is sensor calibration and why is it important in IoT systems? How would you calibrate an analog soil moisture sensor to give accurate percentage readings? Describe the two-point calibration method. (2 marks)

Q30. (Theory) Explain I2C protocol with respect to IoT sensor communication. What are SDA and SCL lines? How does I2C addressing work? Name 3 common IoT sensors that use I2C. (2 marks)

Module 4: Wireless Communication & IoT Protocols

Marks: 20 | Questions: Q31–Q40 | Type: Connectivity Projects



Communication Technologies Covered

- Bluetooth Classic (HC-05) & BLE (ESP32)
- Wi-Fi (ESP8266 / ESP32)
- MQTT Protocol for IoT messaging
- HTTP REST API calls from microcontroller

Section A: Bluetooth Projects (4 marks each)

Q31. Bluetooth LED Controller (HC-05 + Arduino)

Pair HC-05 with an Android phone using a Bluetooth terminal app. Commands received via Bluetooth:

```
'R' → Turn Red LED ON
'G' → Turn Green LED ON
'B' → Turn Blue LED ON
'X' → All LEDs OFF
'F' → Flash all LEDs 5 times
```

Upload code to /week4/bt_led_control/. Include pairing instructions in README (default PIN: 1234 or 0000).

Q32. Bluetooth Sensor Dashboard (ESP32 BLE)

Using ESP32, broadcast DHT11 sensor data over BLE every 3 seconds. A BLE scanning app (nRF Connect or LightBlue) should display: Temperature, Humidity, and a timestamp. Implement GATT characteristics with descriptors. Upload code to /week4/ble_sensor/. Document characteristic UUIDs in README.

Section B: Wi-Fi & Internet (4 marks each)

Q33. Wi-Fi Temperature Monitor (ESP8266/ESP32)

Connect ESP8266 to a Wi-Fi network. Build a simple web server that displays DHT11 data:

- HTML page served at the device's local IP address
- Page shows Temperature, Humidity with auto-refresh every 10 seconds
- Add a button on the web page to toggle an LED on the board

- Display uptime (how long the device has been ON) on the page

Screenshot of the working web page must be uploaded to /week4/wifi_webserver/screenshots/.

Q34. MQTT Sensor Publisher

Use ESP32 to publish DHT11 readings to a free MQTT broker (broker.hivemq.com or test.mosquitto.org):

- Topic: iitjammu/summer26/[yourname]/temperature
- Topic: iitjammu/summer26/[yourname]/humidity
- Publish every 5 seconds in JSON format: {"value": 28.5, "unit": "C", "ts": 1234567890}
- Subscribe to iitjammu/summer26/[yourname]/led_control and toggle LED based on message

Test using MQTT Explorer or MQTT.fx. Submit a screenshot showing messages received.

Appendix A: GitHub Submission Checklist

Before submitting your assignment, verify each item below:

#	Checklist Item	Done? (✓)
1	GitHub account created with real name and profile photo	
2	Repository named: iot-summer-school-2026 (public)	
3	Folder structure: /week1, /week2, /week3, /week4, /final_project, /theory, /portfolio	
4	README.md at root level with student name, roll number, and project overview	
5	Each assignment subfolder has its own README.md	
6	All .ino files are properly commented (author, date, description at top)	
7	.gitignore file excluding build artifacts, .DS_Store, *.hex, *.elf	
8	Minimum 5 Git commits per module with descriptive commit messages	
9	No passwords or API keys committed to the repository	
10	config.h files added to .gitignore where API keys are used	
11	At least one pull request merged (Q7 collaborative exercise)	
12	Screenshot files uploaded for Wi-Fi dashboard (Q33) and MQTT (Q34)	
13	sample_readings.csv and sensor_log.txt uploaded (Q21, Q24)	
14	Circuit diagram / Tinkercad screenshots uploaded (Q23)	
15	Final project proposal and portfolio markdown files uploaded	

Appendix B: Grading Rubric

Assessment Criterion	Excellent (4)	Good (3)	Adequate (2)	Poor (1)
Code Functionality	Works perfectly, handles edge cases	Works with minor issues	Partially working	Does not compile or run
Code Quality	Well-commented, clean, modular	Mostly clean with some comments	Minimal comments, messy	No comments, spaghetti code
GitHub Usage	Excellent commits, branches, PRs	Good commits and structure	Few commits, flat structure	Single commit or private repo
Documentation (README)	Complete, clear, professional	Most sections present	Brief, missing key info	Absent or placeholder
Circuit Correctness	Accurate schematic/diagram	Minor errors in diagram	Conceptually correct only	Absent or incorrect
Innovation / Bonus	All bonus implemented creatively	Some bonus attempted	Basic solution only	Requirements not met



Marks Distribution Summary

- Module 1 – GitHub & Version Control: 20 marks
- Module 2 – Arduino Fundamentals: 20 marks
- Module 3 – Sensors & Actuators: 20 marks
- Module 4 – Wireless Communication: 20 marks
- Module 5 – Advanced Projects: 20 marks

TOTAL: 100 Marks | Bonus marks available up to 10 additional points